MODULE 2 · STATISTICS & PROBABILITY · TOPIC 2.6

The Uniform Distribution

Day 26 — the simplest distribution, and the one everything else is built from



Boston University

2	1	0
Forms: discrete and continuous	Idea that changes everything: area	Random numbers that are not uniform

Prepared & presented by **Srinivasa Sai Chava**

WHY THIS SESSION

A flat line. What could there possibly be to learn?

What it looks like

Every outcome equally likely. A fair die. A random number between 0 and 1.

No peak, no tail, no skew. It is the distribution you draw when you have nothing interesting to say.

What it is actually for

- Where “probability = area” finally makes sense
- The honest model when you know only the bounds
- The raw material for every other distribution
- The thing people assume without checking

Three claims today, in order of how much they will surprise you

1. The probability of any exact value is ZERO
2. The height of the curve can be bigger than 1
3. Every random number your computer makes begins here

WARM-UP • 5 MINUTES

Four questions from Day 25

1	**“Out of” or “per” — what does it decide?**	→ Binomial if there is a countable n; Poisson if it is a rate.
2	**“At least 3” — how do you write it?**	→ 1 − cdf(2). Never 1 − cdf(3).
3	**Poisson mean and variance?**	→ Both equal λ. If var ≫ mean, events are clustering.
4	**Why did both distributions go bell-shaped?**	→ No answer yet — that is Day 27.

Today is the last distribution before the idea that ties all of them together. Keep question 4 in mind — by the end of this session you will have watched it happen a third time.

SESSION PLAN

How today's two hours are structured

1 **Discrete uniform** — 15 min

A die: 1/n, and the two formulas

2 **Continuous uniform** — 30 min

The rectangle, and why area is everything

3 **Is it really uniform?** — 20 min

Testing it, and two famous failures

4 **The foundation** — 20 min

Every random number starts as U(0,1)

5 **Where it leads** — 5 min

One uniform is flat; twelve are a bell

6 **Practice notebook** — 30 min

You code, I walk around

Section 2 is the one to pay attention to. It contains the single idea that separates discrete from continuous probability, and it is the idea people most often carry a wrong version of for years.

1

Discrete Uniform

A die, and the two formulas · 15 minutes

- Every outcome 1/n
- Mean and variance
- Where it genuinely applies

Start with the simplest thing in probability



There is nothing to learn here yet

Six outcomes, each equally likely, so each has probability 1/6.

That is the whole discrete uniform distribution. It has one parameter — how many outcomes — and once you know it there is nothing left to compute.

```
stats.randint(1, 7).pmf(3)

  ->  0.16666666
```

Worth saying out loud: the bars are flat because nothing distinguishes the outcomes. The moment one face is heavier than the others, this stops being the right model — and section 3 is about how you would know.

Mean and spread, for n equally likely values a … b

The mean is the obvious one

$$\text{mean} = (a + b) / 2$$

Dead centre, because the distribution is symmetric about its middle. For a die: (1 + 6) / 2 = 3.5 — a value the die can never actually show.

The variance is the one to look up

$$\text{variance} = (n^2 - 1) / 12$$

Nobody derives this in their head. For a die: (36 − 1) / 12 = 2.9167, so sd = 1.708.

The 12 will turn up again in ten minutes, and again on Day 27.

Check it against scipy rather than trusting either of us

```
d = stats.randint(1, 7)       # a fair die
d.mean()   ->  3.5
d.var()    ->  2.9166666
d.std()    ->  1.7078251
```

A mean the outcome can never equal is not a mistake — it is the long-run average. Roll a die a thousand times and the running average heads for 3.5 without ever landing on it.

This is the same point as “2.4 children per family”.

2

Continuous Uniform

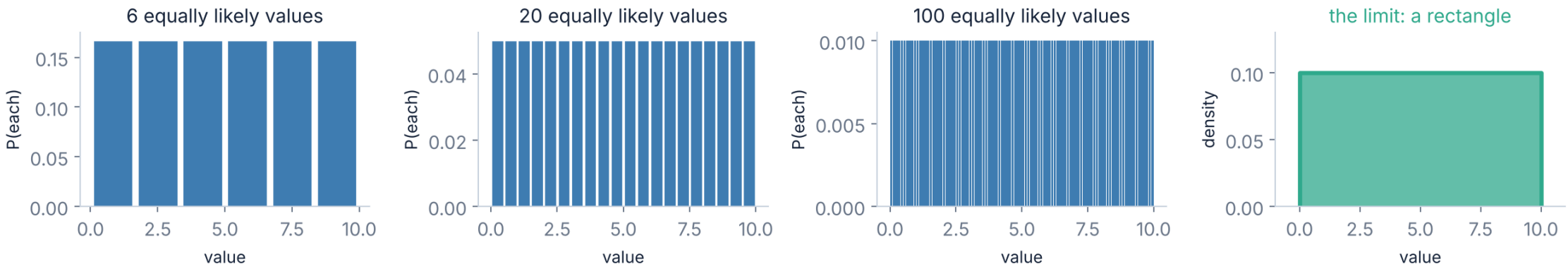
The rectangle, and why area is everything · 30 minutes

- From bars to a rectangle
- Probability is AREA
- Density is not probability
- The cdf

Slides by Srinivasa Sai Chava

What happens when the outcomes get closer together

Squeeze the bars together and the staircase becomes a flat line



Watch the height collapse

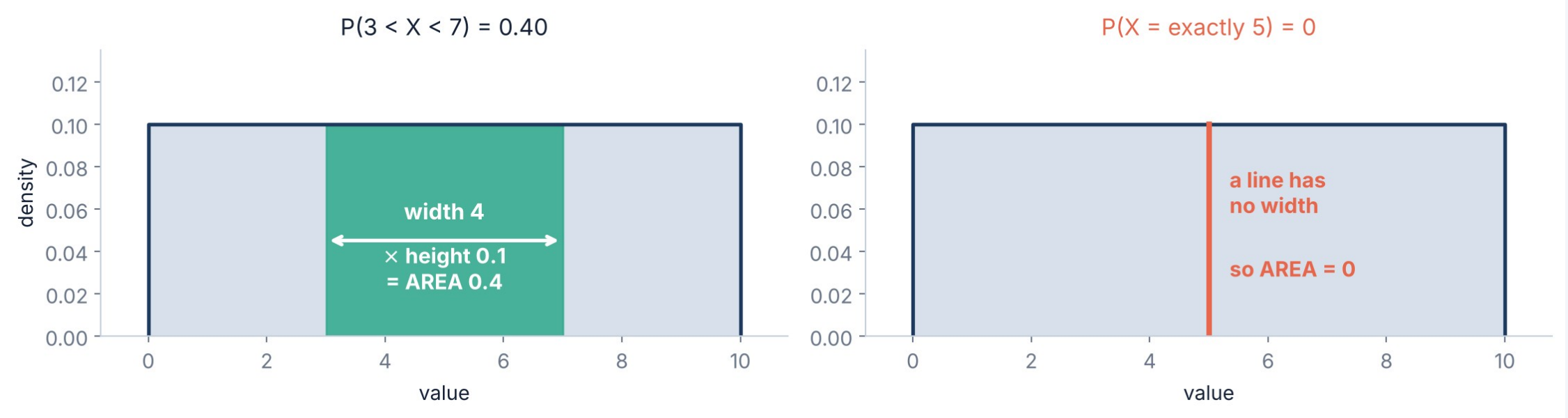
With 6 outcomes each bar is 1/6 tall. With 100, each is 1/100. Keep going and every individual probability goes to zero — but the total stays 1, because there are more of them.

In the limit there are no bars at all

A value can be anything between 0 and 10 — 3, or 3.1, or 3.14159. There are infinitely many, so asking for the probability of one of them stops making sense.

Something has to replace it.

⭐ Probability becomes AREA, not height



A range has an answer

The band from 3 to 7 is 4 wide and 0.1 tall, so its area is 0.4. That is the probability — 40% of values land in there.

A single point does not

“Exactly 5” is a line. A line has zero width, so zero area, so probability exactly 0 — even though 5 is perfectly possible.

“Probability zero” does not mean “impossible”

The objection, which is a good one

“If P(X = 5) = 0 and P(X = 6) = 0 and so on for every value — then nothing can happen. But something does happen every time I draw a number. How?”

This is worth taking seriously. It is the point where continuous probability stops matching everyday intuition.

The resolution

Something does happen — but you never observe it to infinite precision. Your measurement said 5, meaning somewhere in 4.995 to 5.005, and THAT has probability 0.001.

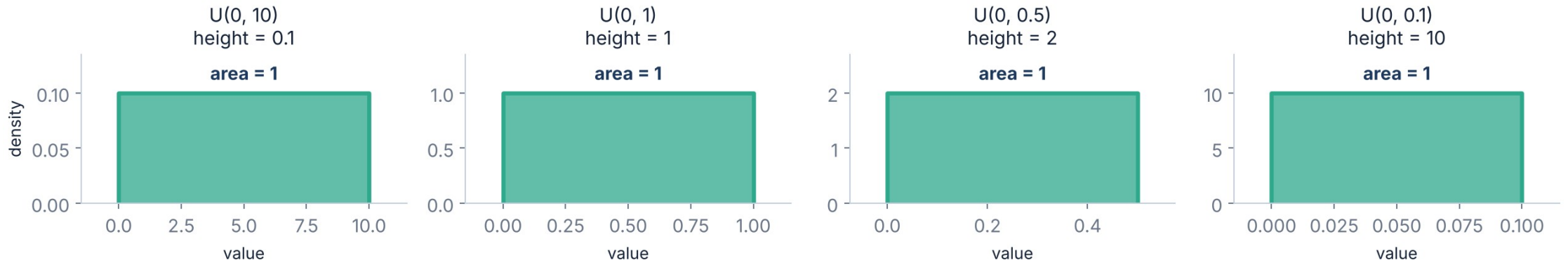
Every real observation is a range. “Exactly 5” is an idealisation that no instrument has ever reported.

The practical rule

For discrete data, ask “what is the probability of exactly 3?” — it has a real answer. For continuous data, never ask that; ask about a range instead. If your code computes a probability for a continuous variable and gets exactly 0.0, check whether you asked for a point.

⚠️ And the height can be bigger than 1

All four have area 1. The height is a DENSITY, not a probability — it can be any size.



A probability can never exceed 1. A density can.

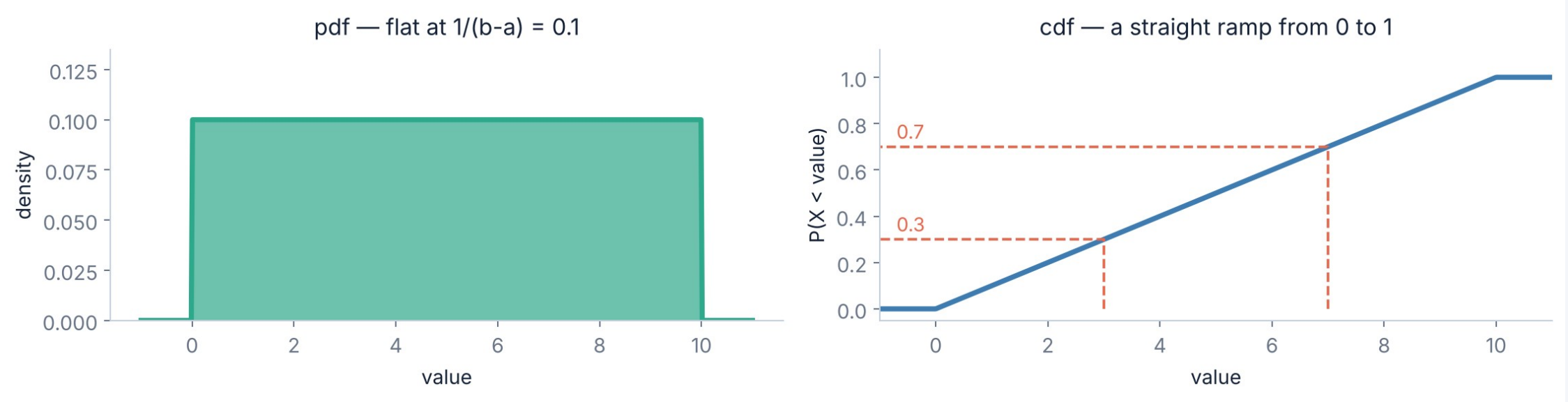
Squeeze U(0,1) down to U(0,0.5) and the same amount of probability has to fit into half the width — so the height doubles to 2. Narrow it to U(0,0.1) and the height is 10.

What the number actually means

A density is probability PER UNIT of x — like a speed, or a price per kilo. 10 per unit over a width of 0.1 gives 1. Nothing is broken.

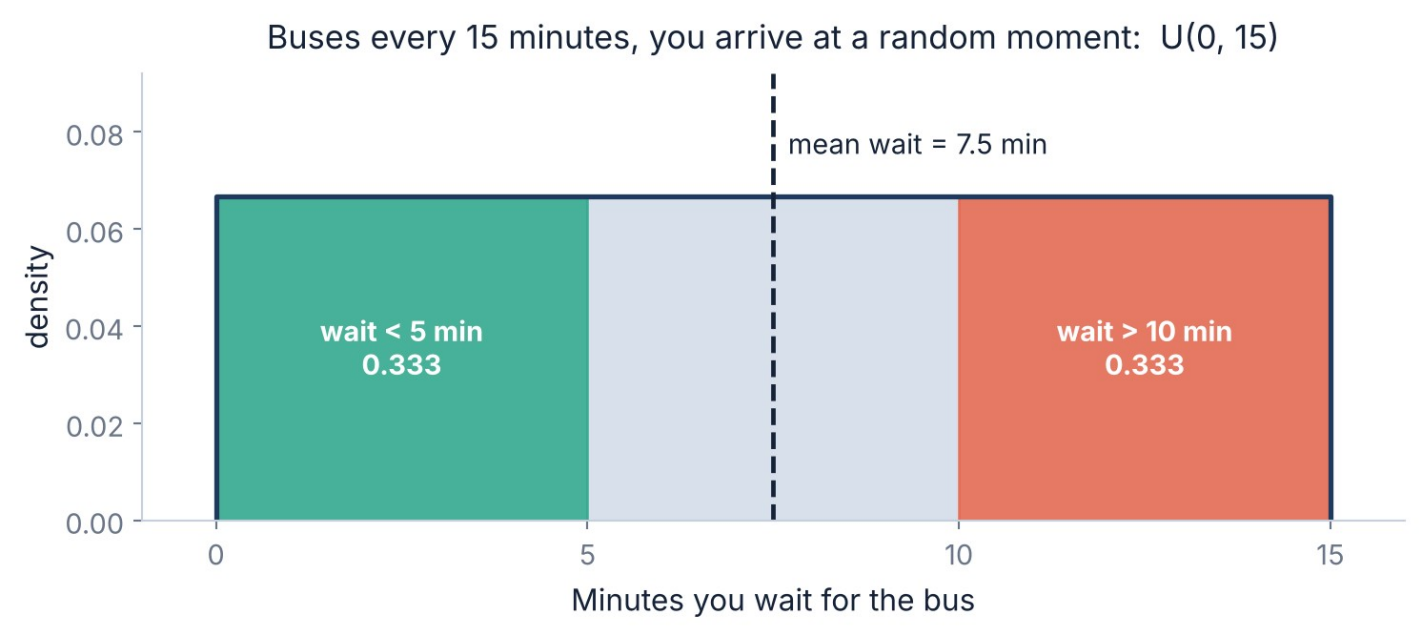
So pdf() never returns a probability. Use cdf() for that.

The two curves, and which one answers questions



The question	Code	Answer
P(X < 3)	`uniform.cdf(3, 0, 10)`	0.30
P(3 < X < 7)	`uniform.cdf(7,0,10) - uniform.cdf(3,0,10)`	0.40
P(X = exactly 5)	`it is always 0 — do not ask`	0.00

Worked through: how long will you wait for the bus?



Why this is uniform

Buses every 15 minutes. You turn up without checking the timetable, so your arrival is equally likely at any point in the gap.

That makes your wait U(0, 15).

Question	Answer
mean wait	7.5 min
sd	4.33 min
P(wait < 5)	0.333
P(wait > 10)	0.333

The useful version of this: a third of the time you wait under 5 minutes and a third of the time you wait over 10. Quoting “an average wait of 7.5 minutes” hides both. It is the same lesson as yesterday's call centre — an average describes the typical case and says nothing about the cases that annoy people.

The continuous formulas, and the 12 that keeps appearing

$$\text{mean} = (a + b) / 2$$

$$\text{variance} = (b - a)^2 / 12$$

For the bus: mean (0+15)/2 = 7.5, variance 225/12 = 18.75, sd 4.33.

Where the 12 comes from

It falls out of integrating x^2 across the rectangle. You will not need to do that — but do remember the consequence:

the sd of U(0,1) is $1/\sqrt{12} = 0.2887$, a number you will meet again the moment anyone adds uniforms together.

⚠️ The scipy trap that will catch you at least once

```
stats.uniform(5, 10)   # you probably meant U(5, 10)
   .ppf([0, 1])  ->  [5., 15.]   # it is U(5, 15)

stats.uniform(loc=5, scale=5)  ->  U(5, 10)  # correct
```

scipy takes loc and SCALE, not the two endpoints. The scale is the WIDTH.

So U(a, b) is uniform(a, b - a). Getting this wrong gives a distribution with the right lower bound and a silently wrong upper one — and a mean that is out by half the width.

3

Is It Really Uniform?

Testing the assumption · 20 minutes

- When it is right
- Testing a die
- How much data you need
- Two famous failures

Uniform is a real model, and also a lazy default

✓ Genuinely uniform

Rounding error

Round to the nearest 0.1 and the error really is U(−0.05, 0.05). Provably, not approximately.

A fair mechanism

A fair die, a well-shuffled deck, a lottery draw — designed to be uniform.

Arrival within a known gap

You turn up without a timetable: your position in the gap is uniform.

“All I know is the bounds”

The honest model when you know the range and nothing else.

× Assumed, and wrong

Exam marks across 0–100

Marks cluster. The bounds are not the distribution.

Birthdays across the year

September is busier than February, and not only because it is longer.

Leading digits of real numbers

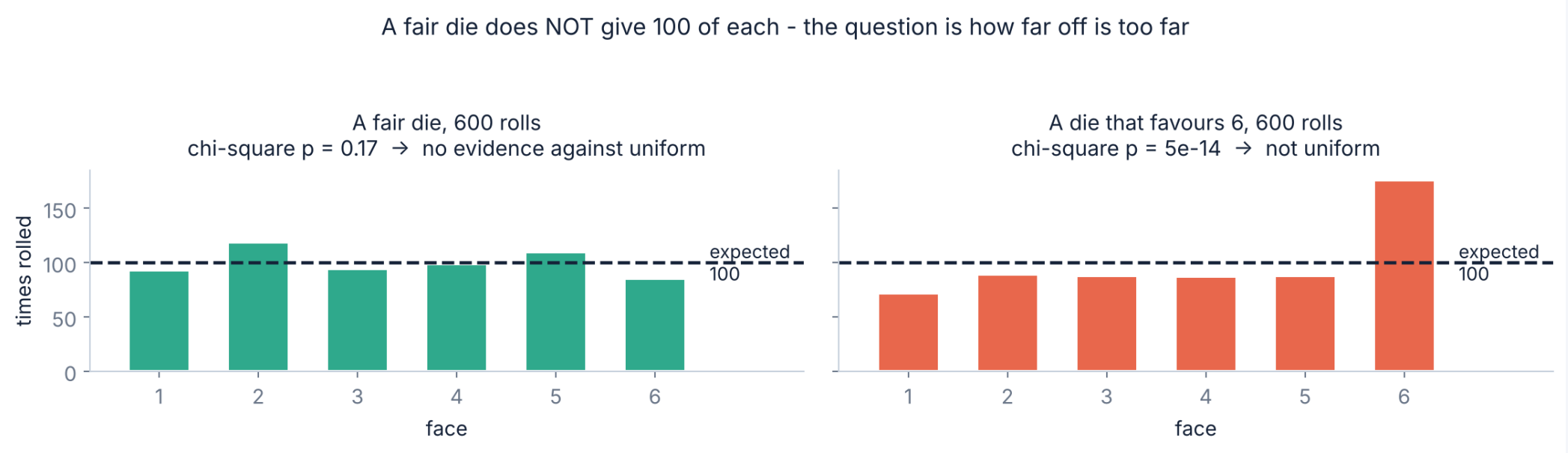
Nowhere near uniform — we look at this shortly.

“Customers arrive uniformly”

Almost never. Lunchtime exists. Use Poisson with a varying rate.

The pattern: uniform is right when something MAKES it uniform. It is wrong whenever it was chosen because nobody looked.

How would you know if a die were loaded?



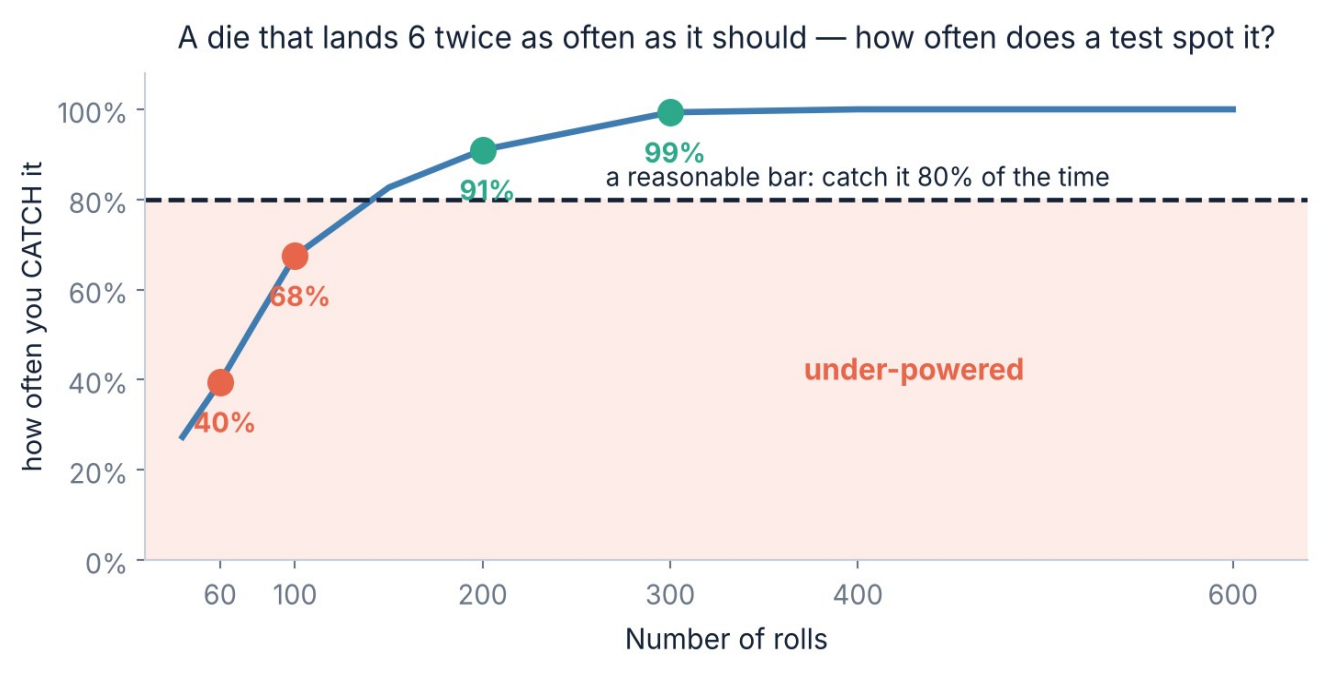
A fair die does not give 100 of each

The fair die above ranged from 85 to 119. That spread is normal. So “the counts are uneven” is not evidence of anything.

The chi-square test asks how far is too far

```
stats.chisquare(counts)
  fair   -> p = 0.17   no evidence
  loaded -> p = 5e-14  not uniform
```

How many rolls before you could catch it?



A badly loaded die

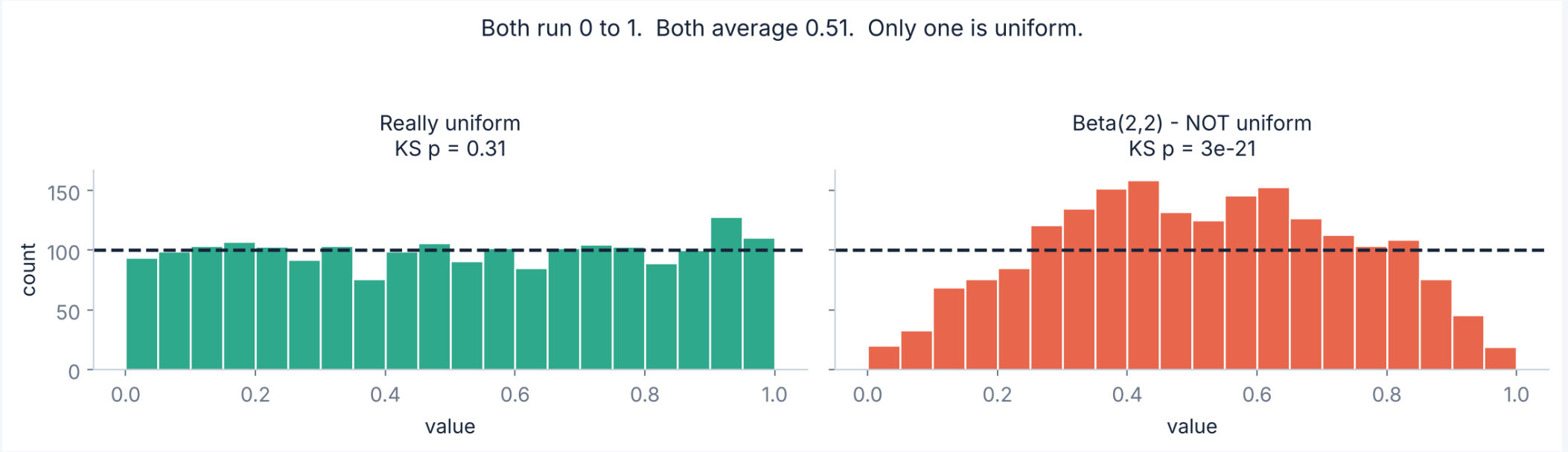
This die lands on 6 twice as often as it should — 28.6% instead of 16.7%. That is blatant cheating.

With 60 rolls a chi-square test spots it only 40% of the time. You would miss it more often than you caught it.

200 rolls: 91%. 300 rolls: 99%.

The lesson, which is the same one as yesterday's A/B test: "I tested it and found nothing" is only meaningful if you say how much data you had. With 60 rolls you would have missed this blatant cheat 6 times in 10 — so finding nothing told you almost nothing.

Same range, same mean, and not uniform at all



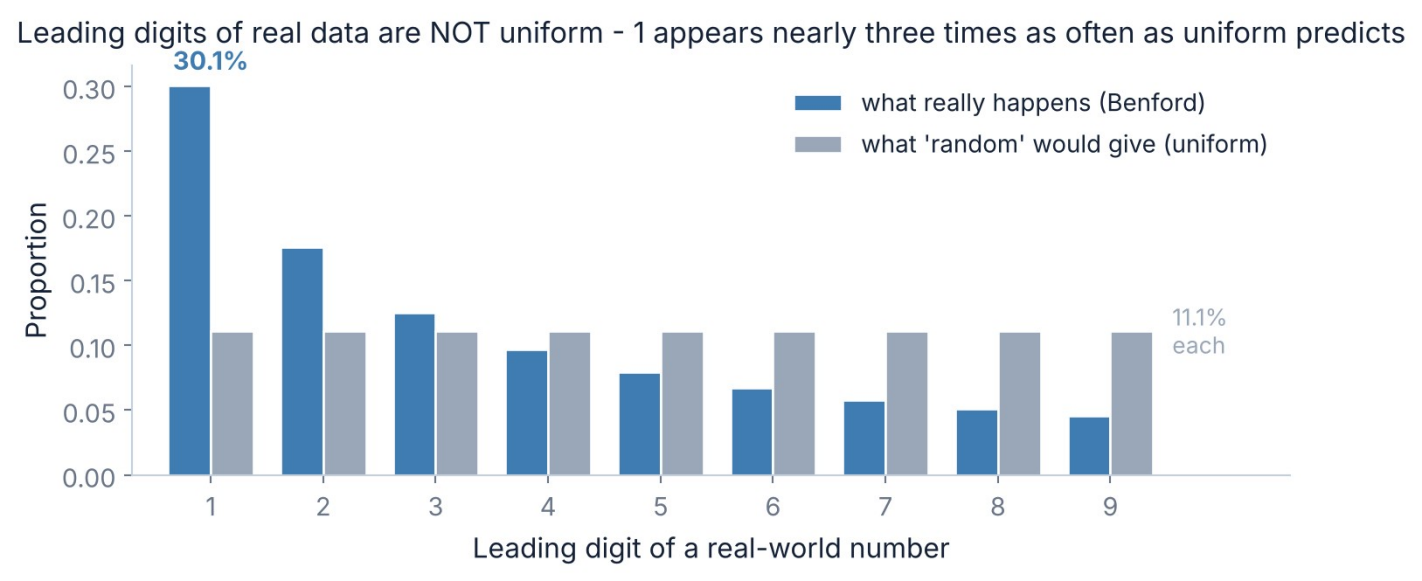
What does NOT prove uniformity

Both samples run from 0 to 1. Both average about 0.51. Checking the bounds and the mean tells you nothing — the shape in between is the whole question.

What does: the KS test, or just look

```
stats.kstest(data, 'uniform')
  really uniform -> p = 0.31
  beta(2,2)      -> p = 3e-21
```

The most famous thing that is not uniform



Why this happens

Real quantities — revenues, populations, river lengths — spread across orders of magnitude. Something growing from 100 to 200 doubles; from 900 to 1000 it rises 11%.

So a value spends far longer with a leading 1 than a leading 9.

$$P(d) = \log_{10}(1 + 1/d)$$

This is used to detect fraud. People inventing numbers spread their leading digits roughly evenly, because that is what "random" feels like. Real accounts do not. Auditors run this check on expense claims and tax returns — a dataset that is suspiciously uniform is the red flag.

4

The Foundation

Every random number starts here · 20 minutes

- What your computer actually makes
- Inverse transform sampling
- Pseudorandom, not random

Your computer can only make one distribution

Underneath, there is only this

```
rng.random()
  ->  0.7340815...     a U(0,1) number
```

That is the only random thing the machine does. Every other distribution you have called this term — normal, binomial, Poisson — is a transformation applied to a stream of these.

Which is a strong claim, so test it

If true, you should be able to build a normal distribution by hand from nothing but rng.random() — no rng.normal anywhere.

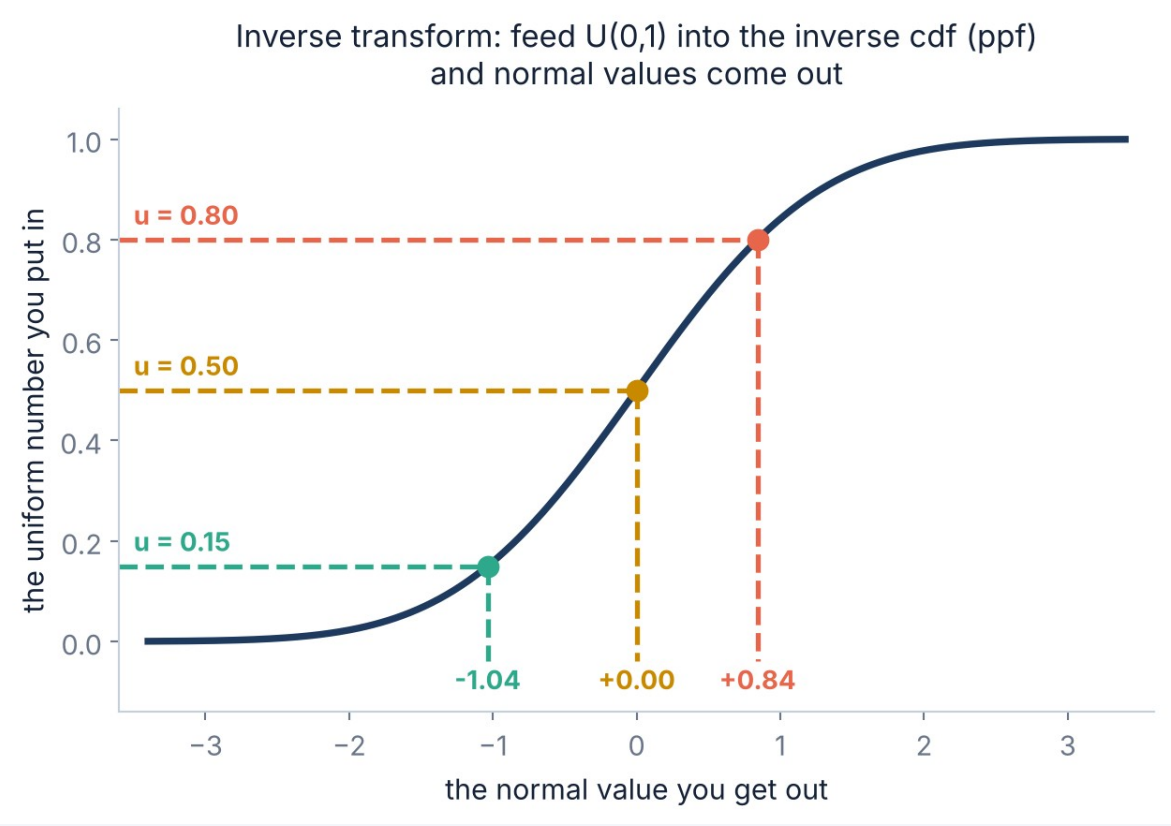
And you can. It takes one line, and it works for every distribution with a cdf you can invert.

That method is next.

Why you should care, beyond the trivia

When you meet a distribution scipy does not implement — a custom one from a paper, or an empirical one you measured yourself — you will need to sample from it. This is how. It is also how every simulation and every bootstrap you write from here on actually works underneath.

Feed a uniform number into the inverse cdf



Read it from the left

Pick a number between 0 and 1 — that is the vertical axis. Go across to the cdf curve, then drop down. The value you land on is your sample.

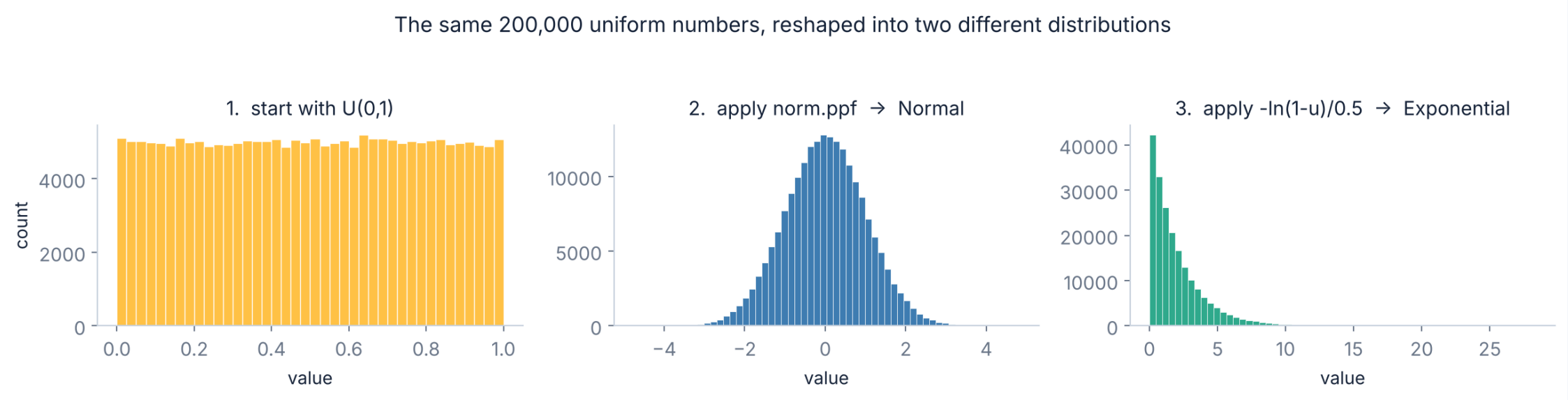
u = 0.50 gives 0.00, the middle. u = 0.80 gives +0.84.

Why it works

Where the curve is STEEP, a small slice of the vertical axis maps to a narrow range of x — so values pile up there. Where it is FLAT, the same slice spreads across a wide range, so values are rare.

Steep means likely. That is exactly what a density is.

The same uniform numbers, reshaped twice




```
u = rng.random(200_000)      # ONE source

normal = stats.norm.ppf(u)
expon  = -np.log(1 - u) / 0.5
```

No rng.normal was called

The normal came out with mean −0.001 and sd 1.001. The exponential came out with mean 1.999 against a target of 2. Both built from a flat line.

⚠️ None of this is actually random

The same seed gives the same “random” numbers

```
np.random.default_rng(42).random(3)
  ->  [0.7739, 0.4388, 0.8585]

#  run it again. Same three numbers. Always.
```

It is a deterministic formula that produces a sequence which PASSES the uniformity tests. Statistically uniform, not unpredictable.

Which is a feature, not a bug

- Your results reproduce — yours and everyone else's
- A failing test fails the same way twice
- A reviewer can rerun your notebook and get your numbers
- Every figure in this course is seeded for exactly this reason

The one place it matters that these are predictable

Never use numpy.random or Python's random module for anything security-related — passwords, tokens, keys. Given enough output, the sequence can be worked backwards and every future value predicted. Use the secrets module instead. It is slower, and it is not reproducible, and both of those are the point.

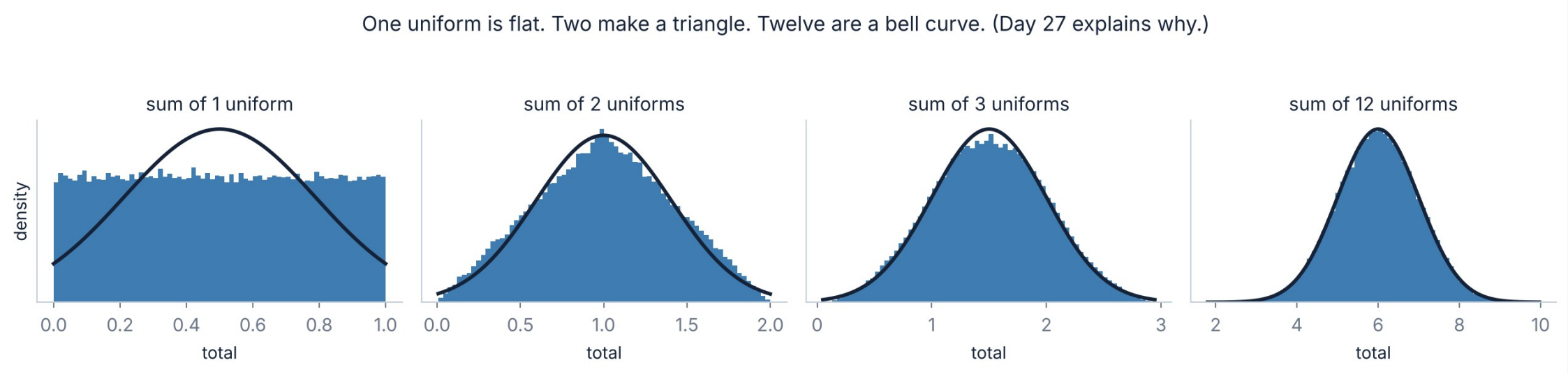
5

Where It Leads

The last step before Day 27 · 5 minutes

- One uniform is flat
- Twelve are a bell curve

The flattest distribution there is, added to itself



Nothing about a flat line suggests a bell curve

One uniform is flat. Two added together make a triangle — totals near the middle have many ways to happen and totals near the ends have few. Three start to curve. Twelve are indistinguishable from a normal curve.

That is the third time this has happened: the binomial did it as n grew, the Poisson did it as λ grew, and now the flattest distribution of all does it too. Day 27 explains why it is not a coincidence.

Module 2 so far — four distributions

	Normal	Binomial	Poisson	Uniform
Day	24	25	25	26
Type	continuous	discrete	discrete	either
Answers	how big?	how many out of n?	how many per interval?	equally likely
Parameters	μ, σ	n, p	λ	a, b
Mean	μ	n × p	λ	(a + b) / 2
Variance	σ^2	np(1−p)	λ	$(b - a)^2 / 12$
Shape	bell	peak at np	right-skewed	flat
scipy	`norm`	`binom`	`poisson`	`uniform / randint`

The continuous ones use pdf and cdf; the discrete ones use pmf and cdf. uniform takes loc and WIDTH, not the two endpoints.

RECAP

Debug corner — eight uniform mistakes

You did	What goes wrong	Fix
uniform(5, 10) for U(5, 10)	scipy takes loc and WIDTH: you got U(5, 15)	uniform(5, 5), or uniform(a, b-a)
read pdf() as a probability	it can exceed 1 and is not a probability	cdf for probabilities, pdf for density
asked P(X = 4.7) on continuous data	always exactly 0, silently	ask about a range
randint(1, 6) for a six-sided die	high is EXCLUSIVE: you built a 5-sided die	randint(1, 7)
random.randint vs np.integers	one includes the top, one does not	check which library you are in
assumed uniform without looking	same range and mean, wrong shape	histogram it, then chisquare or kstest
‘the test found nothing, so it is fair’	60 rolls miss a blatant cheat 6 times in 10	say how much data you had
used np.random for a token	the sequence is predictable	the secrets module

Row 1 is the one that will silently change your answer. Row 4 is the one that will change it by exactly one sixth and look plausible.

Putting it together — sample any distribution you like

```python
import numpy as np

#  GOAL: draw from a loaded die using ONLY rng.random()
#  the die should land on 6 half the time

rng   = np.random.default_rng(26)
probs = np.array([.1, .1, .1, .1, .1, .5])

#  --- 1. the cdf: cumulative probability of each face
cum = np.cumsum(probs)
print(cum)       # [.1 .2 .3 .4 .5 1.]

#  --- 2. one uniform number per roll
u = rng.random(200_000)

#  --- 3. find which band each number landed in
rolls = np.searchsorted(cum, u) + 1

#  --- 4. did it work?
freq = np.bincount(rolls, minlength=7)[1:] / len(rolls)
print(freq.round(4))
#  [0.1000 0.1006 0.0988 0.1010 0.0999 0.4996]

#  no rng.choice, no rng.multinomial - just a flat line
#  and a cumulative sum.
```

This IS inverse transform

searchsorted is the discrete ppf

The band widths ARE the probabilities

6 owns half the interval

cumsum must end at 1.0

otherwise some u lands nowhere

+1 because faces start at 1

searchsorted returns 0-based

200,000 draws to see 4 decimals

fewer and the check is meaningless

Works for ANY probabilities

change the array, nothing else

This is roughly what rng.choice(p=...) does internally. Writing it once means it is no longer magic.

OVER TO YOU

Practice session — 30 minutes

A **Predict the output** — 8 questions

pdf vs cdf, loc/scale, randint bounds

B **Spot & fix the bug** — 4 questions

The width trap, a point probability, a bad test

C **Write the code** — 5 tasks

Model it, test it, decide

D **Challenge** — 2 tasks

Build a sampler, and audit a dataset

Your notebook

`Day26_Practice_Questions.ipynb`

- Is it continuous or discrete?
- Width or endpoint?
- Histogram before you test
- Solutions shared after class

Part D asks you to audit a dataset for uniformity. There is no "right answer" printed anywhere — you have to decide, and say what would change your mind.

TODAY IN TWELVE LINES

Recap

1. Uniform means every outcome equally likely — no peak, no skew.
2. Discrete: n outcomes, each 1/n. Mean (a+b)/2, variance $(n^2-1)/12$.
3. Continuous: a flat rectangle of height 1/(b−a) over a to b.
4. ⭐ Probability is AREA, not height. Width × height.
5. So P(X = any exact value) = 0 — which is not the same as impossible.
6. ⚠️ A density can exceed 1. U(0, 0.1) has height 10, and area 1.
7. Continuous mean (a+b)/2, variance $(b-a)^2/12$. sd of U(0,1) is 0.2887.
8. ⚠️ scipy takes loc and WIDTH: U(a,b) is uniform(a, b−a).
9. Same range and same mean do NOT make it uniform. Look at the shape.
10. Test it: chisquare for discrete, kstest for continuous — and say your n.
11. Every random number starts as U(0,1); ppf reshapes it into anything.
12. One uniform is flat. Twelve added together are a bell curve.

Homework

Build a sampler for a distribution scipy does not have, using only rng.random() · Take any numeric column from your Day 21 project and test whether its leading digits follow Benford · Find one place in your own code where you assumed uniform without checking

Next class — Topic 2.7

The Central Limit Theorem. Three times now a distribution has turned into a bell curve for no apparent reason. Next class is the one that explains it — and it is the result the whole of the rest of statistics rests on.